

C++ Standard Library Issues to be moved in Varna 2023

Doc. no. P2910R0

Date: 2023-06-09

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

- [2994](#). Needless UB for `basic_string` and `basic_string_view`
- [3884](#). `flat_foo` is missing allocator-extended copy/move constructors
- [3885](#). 'op' should be in `[zombie.names]`
- [3887](#). Version macro for `allocate_at_least`
- [3893](#). LWG 3661 broke `atomic<shared_ptr<T>> a; a = nullptr;`
- [3894](#). `generator::promise_type::yield_value(ranges::elements_of<Rng, Alloc>)` should not be `noexcept`
- [3903](#). `span` destructor is redundantly `noexcept`
- [3904](#). `lazy_split_view::outer_iterator`'s const-converting constructor isn't setting `trailing_empty_`
- [3905](#). Type of `std::fexcept_t`
- [3912](#). `enumerate_view::iterator::operator-` should be `noexcept`
- [3914](#). Inconsistent *template-head* of `ranges::enumerate_view`
- [3915](#). Redundant paragraph about expression variations
- [3925](#). Concept `formattable`'s definition is incorrect
- [3927](#). Unclear preconditions for `operator[]` for sequence containers
- [3935](#). `template<class X> constexpr complex& operator=(const complex<X>&)` has no specification
- [3938](#). Cannot use `std::expected` monadic ops with move-only `error_type`
- [3940](#). `std::expected<void, E>::value()` also needs `E` to be copy constructible

[2994](#). Needless UB for `basic_string` and `basic_string_view`

Section: 23.2 [\[char.traits\]](#), 23.4.3.2 [\[string.require\]](#), 23.3.3 [\[string.view.template\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Gennaro Prota **Opened:** 2017-07-03 **Last modified:** 2023-06-01

Priority: 3

View all other [issues](#) in `[char.traits]`.

Discussion:

`basic_string` and `basic_string_view` involve undefined behavior in a few cases where it's simple for the implementation to add a `static_assert` and make the program ill-formed.

With regards to `basic_string`, 23.2 [\[char.traits\]](#)/3 states:

`Traits::char_type` shall be the same as `CharT`.

Here, the implementation can add a `static_assert` using the `is_same` type trait. Similar issues exist in 23.4.3.2 [\[string.require\]](#) and, for `basic_string_view`, in 23.3.3 [\[string.view.template\]](#)/1.

[2017-07 Toronto Tuesday PM issue prioritization]

Priority 3; need to check general container requirements

Partially by the adoption of [P1148](#) in San Diego.

Tim opines: "the remainder deals with allocator value type mismatch, which I think is NAD."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Edit 23.2 [\[char.traits\]](#) as indicated:

-3- To specialize those templates to generate a string or istream class to handle a particular character container type `CharT`, that and its related character traits class `Traits` are passed as a pair of parameters to the string or istream template as parameters `charT` and `traits`. ~~If `Traits::char_type` shall be the same~~ **is not the same type as `CharT`, the program is ill-formed.**

2. Edit 23.4.3.2 [\[string.require\]](#) as indicated:

-3- In every specialization `basic_string<charT, traits, Allocator>`, ~~if the type `allocator_traits<Allocator>::value_type` shall name the same type~~ **is not the same type as `charT`, the program is ill-formed.** Every object of type `basic_string<charT, traits, Allocator>` shall use an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used shall be obtained as described in 24.2.2.1 [\[container.requirements.general\]](#). In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall satisfy the character traits requirements (23.2 [\[char.traits\]](#)). ~~If, and the type `traits::char_type` shall name the same type~~ **is not the same type as `charT`, the program is ill-formed.**

3. Edit 23.3.3 [\[string.view.template\]](#) as indicated:

-1- In every specialization `basic_string_view<charT, traits>`, the type `traits` shall satisfy the character traits requirements (23.2 [\[char.traits\]](#)). ~~If, and the type `traits::char_type` shall name the same type~~ **is not the same type as `charT`, the program is ill-formed.**

[2023-05-15; Jonathan Wakely provides improved wording]

As noted above, most of this issue was resolved by [P1148R0](#). The remainder is the change to make it ill-formed if the allocator has the wrong `value_type`, but since [P1463R1](#) that is already part of the Allocator-aware container requirements, which apply to `basic_string`.

[2023-06-01; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

There was a request to editorially move the added text in *Note 1* into *Note 2* after this is approved.

Proposed resolution:

This wording is relative to [N4950](#).

1. Edit 23.4.3.2 [\[string.require\]](#) as indicated:

-3- ~~In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT`.~~ Every object of type `basic_string<charT, traits, Allocator>` shall use an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. ~~The~~ In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall satisfy the character traits requirements (23.2 [\[char.traits\]](#)).

[*Note 1*: Every specialization `basic_string<charT, traits, Allocator>` is an allocator-aware container, but does not use the allocator's construct and destroy member functions (24.2.2.1 [\[container.requirements.general\]](#)). **The program is ill-formed if `Allocator::value_type` is not the same type as `charT`.** — end note]

[*Note 2*: The program is ill-formed if `traits::char_type` is not the same type as `charT`. — end note]

[3884](#). `flat_foo` is missing allocator-extended copy/move constructors

Section: 24.6.9 [\[flat.map\]](#), 24.6.10 [\[flat.multimap\]](#), 24.6.11 [\[flat.set\]](#), 24.6.12 [\[flat.multiset\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Arthur O'Dwyer **Opened:** 2023-02-06 **Last modified:** 2023-03-23

Priority: Not Prioritized

View other [active issues](#) in [flat.map].

View all other [issues](#) in [flat.map].

Discussion:

This issue is part of the "flat_foo" sequence, LWG [3786](#), [3802](#), [3803](#), [3804](#). flat_set, flat_multiset, flat_map, and flat_multimap all have implicitly defaulted copy and move constructors, but they lack allocator-extended versions of those constructors. This means that the following code is broken:

```
// https://godbolt.org/z/qezv5rTrW
#include <cassert>
#include <flat_set>
#include <memory_resource>
#include <utility>
#include <vector>

template<class T, class Comp = std::less<T>>
using pmr_flat_set = std::flat_set<T, Comp, std::pmr::vector<T>>;

int main() {
    std::pmr::vector<pmr_flat_set<int>> vs = {
        {1,2,3},
        {4,5,6},
    };
    std::pmr::vector<int> v = std::move(vs[0]).extract();
    assert(v.get_allocator().resource() == std::pmr::get_default_resource());
}
```

pmr_flat_set<int> advertises that it "uses_allocator" std::pmr::polymorphic_allocator<int>, but in fact it lacks the allocator-extended constructor overload set necessary to interoperate with std::pmr::vector.

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.6.9.2 [\[flat.map.defn\]](#) as indicated:

```
namespace std {
    template<class Key, class T, class Compare = less<Key>,
    class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
    class flat_map {
    public:
        [...]
        // 24.6.9.3 \[flat.map.cons\], construct/copy/destroy
        flat_map() : flat_map(key_compare()) { }

        template<class Allocator>
        flat_map(const flat_map&, const Allocator& a);
        template<class Allocator>
        flat_map(flat_map&&, const Allocator& a);

        [...]
    };
    [...]
}
```

2. Modify 24.6.9.3 [\[flat.map.cons\]](#) as indicated:

[Drafting note: As a drive-by fix, a missing closing > is inserted in p11.]

```
template<class Allocator>
    flat_map(const flat_map&, const Allocator& a);
template<class Allocator>
```

```

flat_map(flat_map&&, const Allocator& a);
template<class Allocator>
    flat_map(const key_compare& comp, const Allocator& a);
template<class Allocator>
    explicit flat_map(const Allocator& a);
template<class InputIterator, class Allocator>
    flat_map(InputIterator first, InputIterator last, const key_compare& comp, const Allocator& a);
template<class InputIterator, class Allocator>
    flat_map(InputIterator first, InputIterator last, const Allocator& a);
[...]
```

-11- *Constraints:* `uses_allocator_v<key_container_type, Allocator>` is true and `uses_allocator_v<mapped_container_type, Allocator>` is true.

-12- *Effects:* Equivalent to the corresponding non-allocator constructors except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

3. Modify 24.6.10.2 [\[flat.multimap.defn\]](#) as indicated:

```

namespace std {
    template<class Key, class T, class Compare = less<Key>,
            class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
        class flat_multimap {
        public:
            [...]
            // 24.6.10.3 \[flat.multimap.cons\], construct/copy/destroy
            flat_multimap() : flat_multimap(key_compare()) { }

            template<class Allocator>
                flat_multimap(const flat_multimap&, const Allocator& a);
            template<class Allocator>
                flat_multimap(flat_multimap&&, const Allocator& a);

            [...]
        };
    [...]
}
```

4. Modify 24.6.10.3 [\[flat.multimap.cons\]](#) as indicated:

```

template<class Allocator>
    flat_multimap(const flat_multimap&, const Allocator& a);
template<class Allocator>
    flat_multimap(flat_multimap&&, const Allocator& a);
template<class Allocator>
    flat_multimap(const key_compare& comp, const Allocator& a);
[...]
```

-11- *Constraints:* `uses_allocator_v<key_container_type, Allocator>` is true and `uses_allocator_v<mapped_container_type, Allocator>` is true.

-12- *Effects:* Equivalent to the corresponding non-allocator constructors except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

5. Modify 24.6.11.2 [\[flat.set.defn\]](#) as indicated:

```

namespace std {
    template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
        class flat_set {
        public:
            [...]
            // 24.6.11.3 \[flat.set.cons\], constructors
            flat_set() : flat_set(key_compare()) { }

            template<class Allocator>
                flat_set(const flat_set&, const Allocator& a);
            template<class Allocator>
                flat_set(flat_set&&, const Allocator& a);

            [...]
        };
    [...]
}
```

6. Modify 24.6.11.3 [\[flat.set.cons\]](#) as indicated:

```
template<class Allocator>
    flat_set(const flat_set&, const Allocator& a);
template<class Allocator>
    flat_set(flat_set&&, const Allocator& a);
template<class Allocator>
    flat_set(const key_compare& comp, const Allocator& a);
[...]
```

-9- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-10- *Effects*: Equivalent to the corresponding non-allocator constructors except that `c` is constructed with uses-allocator construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

7. Modify 24.6.12.2 [\[flat.multiset.defn\]](#) as indicated:

```
namespace std {
    template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
    class flat_multiset {
    public:
        [...]
        // 24.6.12.3 \[flat.multiset.cons\], constructors
        flat_multiset() : flat_multiset(key_compare()) { }

        template<class Allocator>
            flat_multiset(const flat_multiset&, const Allocator& a);
        template<class Allocator>
            flat_multiset(flat_multiset&&, const Allocator& a);

        [...]
    };
    [...]
}
```

8. Modify 24.6.12.3 [\[flat.multiset.cons\]](#) as indicated:

```
template<class Allocator>
    flat_multiset(const flat_multiset&, const Allocator& a);
template<class Allocator>
    flat_multiset(flat_multiset&&, const Allocator& a);
template<class Allocator>
    flat_multiset(const key_compare& comp, const Allocator& a);
[...]
```

-9- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-10- *Effects*: Equivalent to the corresponding non-allocator constructors except that `c` is constructed with uses-allocator construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

[3885](#). 'op' should be in [\[zombie.names\]](#)

Section: 16.4.5.3.2 [\[zombie.names\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2023-02-11 **Last modified:** 2023-03-22

Priority: Not Prioritized

Discussion:

C++98 and C++11 defined a protected member `std::bind1st::op` so 'op' should still be a reserved name, just like 'bind1st' itself.

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 16.4.5.3.2 [\[zombie.names\]](#) as indicated:

-2- The following names are reserved as members for previous standardization, and may not be used as a name for object-like macros in portable code:

(2.1) — argument_type,

[...]

(2.3) — io_state,

(2.?) — op,

(2.4) — open_mode,

[...]

[3887](#). Version macro for `allocate_at_least`

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2023-02-14 **Last modified:** 2023-03-22

Priority: Not Prioritized

View other [active issues](#) in `[version.syn]`.

View all other [issues](#) in `[version.syn]`.

Discussion:

In Issaquah, we just adopted [P2652R0](#), forbidding user specialization of `allocator_traits`.

It looks like we did not deem that worthy of a feature macro.

However, it also changed the interface of the C++23 addition, `allocate_at_least`, which is covered by the macro `__cpp_lib_allocate_at_least`.

I believe we should have updated that macro for this significant change in how that function is accessed (from free function to a member of `allocator_traits`). Unfortunately, there are no more meetings to process a comment to that effect, and I suspect this rises beyond the purview of an editorial change, although I live in hope (as the original paper left the value of the macro to the editor, although we approved existing working papers where that macro does have a value, i.e., status quo).

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll. But "if we don't get the editors to do it for C++23 there doesn't seem to be any point doing it."

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, as indicated and replace the placeholder `YYYYMM` by the year and month of adoption of [P2652R0](#):

```
[...]  
#define __cpp_lib_algorithm_iterator_requirements 202207L  
    // also in <algorithm>, <numeric>, <memory>  
#define __cpp_lib_allocate_at_least 202106YYYYMML // also in <memory>  
#define __cpp_lib_allocator_traits_is_always_equal 201411L  
    // also in [...]  
[...]
```

[3893](#). LWG 3661 broke `atomic<shared_ptr<T>> a; a = nullptr;`

Section: 33.5.8.7.2 [[util.smartptr.atomic.shared](#)] **Status:** [Tentatively Ready](#) **Submitter:** Zachary Wassall **Opened:** 2023-02-22 **Last modified:** 2023-03-22

Priority: Not Prioritized

View all other [issues](#) in [[util.smartptr.atomic.shared](#)].

Discussion:

LWG [3661](#), "constinit atomic<shared_ptr<T>> a(nullptr); should work" added the following to atomic<shared_ptr<T>>:

```
constexpr atomic(nullptr_t) noexcept : atomic() { }
```

I believe that doing so broke the following example:

```
atomic<shared_ptr<T>> a;  
a = nullptr;
```

For reference, atomic<shared_ptr<T>> provides two assignment operator overloads:

```
void operator=(const atomic&) = delete;           // #1  
void operator=(shared_ptr<T> desired) noexcept;  // #2
```

Prior to LWG [3661](#), the assignment in the example unambiguously matches #2. #1 is not viable because nullptr_t is not convertible to atomic<shared_ptr<T>>. After LWG 3611, #1 is viable and the assignment is ambiguous between #1 and #2.

I believe this could be remedied easily enough by adding an assignment operator to match the added nullptr_t constructor:

```
void operator=(nullptr_t) noexcept;
```

[2023-02-25; Daniel comments and provides wording]

The suggested delegation below to store(nullptr) is not ambiguous and calls the constructor shared_ptr(nullptr_t), which is guaranteed to be no-throwing.

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 33.5.8.7.2 [[util.smartptr.atomic.shared](#)] as indicated:

```
namespace std {  
    template<class T> struct atomic<shared_ptr<T>> {  
        [...]  
        constexpr atomic() noexcept;  
        constexpr atomic(nullptr_t) noexcept : atomic() { }  
        atomic(shared_ptr<T> desired) noexcept;  
        atomic(const atomic&) = delete;  
        void operator=(const atomic&) = delete;  
        [...]  
        void operator=(shared_ptr<T> desired) noexcept;  
        void operator=(nullptr_t) noexcept;  
        [...]  
    private:  
        shared_ptr<T> p; // exposition only  
    };  
}
```

[...]

```
void operator=(shared_ptr<T> desired) noexcept;
```

-5- *Effects:* Equivalent to store(desired).

```
void operator=(nullptr_t) noexcept;
```

-?- Effects: Equivalent to `store(nullptr)`.

[3894](#). `generator::promise_type::yield_value(ranges::elements_of<Rng, Alloc>)` should not be `noexcept`

Section: 26.8.5 [\[coro.generator.promise\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2023-02-25 **Last modified:** 2023-03-22

Priority: Not Prioritized

View other [active issues](#) in `[coro.generator.promise]`.

View all other [issues](#) in `[coro.generator.promise]`.

Discussion:

The overload of `yield_value` for yielding elements of arbitrary ranges does so by creating a nested generator, but to do so it needs to:

- call `ranges::begin/ranges::end` on the range
- allocate a new coroutine frame (unless the allocation is elided by the compiler, which isn't guaranteed)
- copy/move the iterator and sentinel into the coroutine frame

All of these are allowed to throw, so this overload should not be `noexcept`.

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 26.8.5 [\[coro.generator.promise\]](#) as indicated:

```
namespace std {
    template<class Ref, class V, class Allocator>
    class generator<Ref, V, Allocator>::promise_type {
    public:
        [...]
        auto yield_value(const remove_reference_t<yielded>& lval)
            requires is_rvalue_reference_v<yielded> &&
                constructible_from<remove_cvref_t<yielded>, const remove_reference_t<yielded>&>;

        template<class R2, class V2, class Alloc2, class Unused>
        requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
        auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&&, Unused> g) noexcept;

        template<ranges::input_range R, class Alloc>
        requires convertible_to<ranges::range_reference_t<R>, yielded>
        auto yield_value(ranges::elements_of<R, Alloc> r) noexcept;
        [...]
    };
}

[...]
```

```
template<ranges::input_range R, class Alloc>
requires convertible_to<ranges::range_reference_t<R>, yielded>
auto yield_value(ranges::elements_of<R, Alloc> r) noexcept;
```

-13- Effects: Equivalent to:

```
auto nested = [] (allocator_arg_t, Alloc, ranges::iterator_t<R> i, ranges::sentinel_t<R> s)
-> generator<yielded, ranges::range_value_t<R>, Alloc> {
    for (; i != s; ++i) {
        co_yield static_cast<yielded>(*i);
    }
}
```



```
    }  
    };  
    return yield_value(ranges::elements_of(nested(  
        allocator_arg, r.allocator, ranges::begin(r.range), ranges::end(r.range))));  
}
```

[...]

[3903](#). `span` destructor is redundantly `noexcept`

Section: 24.7.2.2.1 [\[span.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Ben Craig **Opened:** 2023-03-11 **Last modified:** 2023-03-22

Priority: Not Prioritized

View other [active issues](#) in [\[span.overview\]](#).

View all other [issues](#) in [\[span.overview\]](#).

Discussion:

The `span` class template synopsis in 24.7.2.2.1 [\[span.overview\]](#) has this declaration:

```
~span() noexcept = default;
```

The `noexcept` is redundant, as `~span` is `noexcept` automatically. I think the entire declaration is unnecessary as well. There is no additional specification for `~span()` at all, much less some that warrants inclusion in the class template synopsis.

Recommended fix:

```
~span() noexcept = default;
```

Alternative fix:

```
~span() noexcept = default;
```

[2023-03-22; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.7.2.2.1 [\[span.overview\]](#), class template `span` synopsis, as indicated:

[...]

```
~span() noexcept = default;
```

[...]

[3904](#). `lazy_split_view::outer_iterator`'s `const`-converting constructor isn't setting `trailing_empty_`

Section: 26.7.16.3 [\[range.lazy.split.outer\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Patrick Palka **Opened:** 2023-03-13 **Last modified:** 2023-05-24

Priority: Not Prioritized

View other [active issues](#) in [\[range.lazy.split.outer\]](#).

View all other [issues](#) in [\[range.lazy.split.outer\]](#).

Discussion:

It seems the const-converting constructor for lazy_split_view's iterator fails to propagate *trailing_empty_* from the argument, which effectively results in the trailing empty range getting skipped over:

```
auto r = views::single(0) | views::lazy_split(0); // r is { {}, {} }
auto i = r.begin();
++i; // i.trailing_empty_ is correctly true
decltype(std::as_const(r).begin()) j = i; // j.trailing_empty_ is incorrectly false
auto k = r.end(); // k.trailing_empty_ is correctly false, and we wrongly have j == k
```

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 26.7.16.3 [\[range.lazy.split.outer\]](#) as indicated:

```
constexpr outer-iterator(outer-iterator<!Const> i)
requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

-4- Effects: Initializes *parent_* with *i.parent_*, and *current_* with `std::move(i.current_)`, and *trailing_empty_* with *i.trailing_empty_*.

[3905](#). Type of `std::fexcept_t`

Section: 28.3.1 [\[cfenv.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Sam Elliott **Opened:** 2023-03-13 **Last modified:** 2023-05-24

Priority: Not Prioritized

Discussion:

`<cfenv>`, as specified in 28.3.1 [\[cfenv.syn\]](#), requires `fexcept_t` to be an integer type:

```
using fexcept_t = integer type;
```

`<cfenv>` was initially added to the (first) Technical Report on C++ Library Extensions via [N1568](#) and then integrated into the C++ Working Draft N2009 in Berlin (April, 2006).

However, C99 does not actually require that `fexcept_t` is an integer type, it only requires:

The type `fexcept_t` represents the floating-point status flags collectively, including any status the implementation associates with the flags.

Relaxing this requirement should not cause conforming C++ implementations to no longer be conforming. In fact, this should enable conforming C implementations to become conforming C++ implementations without an ABI break. The only incompatibility I foresee is where a user's program is initializing a `std::fexcept_t` with an integer value, which would become invalid on some C++ implementations (but not those that were previously conforming).

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 28.3.1 [\[cfenv.syn\]](#), header `<cfenv>` synopsis, as indicated:

```
[...]
namespace std {
    // types
    using fenv_t = object type;
    using fexcept_t = integer object type;
[...]
```

[3912](#). `enumerate_view::iterator::operator-` should be `noexcept`

Section: 26.7.23.3 [\[range.enumerate.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-03-27 **Last modified:** 2023-05-24

Priority: Not Prioritized

View other [active issues](#) in `[range.enumerate.iterator]`.

View all other [issues](#) in `[range.enumerate.iterator]`.

Discussion:

The distance between two `enumerate_view::iterator` is calculated by subtracting two integers, which never throws.

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 26.7.23.3 [\[range.enumerate.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
        requires range-with-movable-references<V>
        template<bool Const>
        class enumerate_view<V>::iterator {
            [...]
        public:
            [...]
            friend constexpr difference_type operator-(const iterator& x, const iterator& y) noexcept;
            [...]
        }
        [...]
}

[...]

friend constexpr difference_type operator-(const iterator& x, const iterator& y) noexcept;

-19- Effects: Equivalent to: return x.pos_ - y.pos_;
```

[3914](#). Inconsistent *template-head* of `ranges::enumerate_view`

Section: 26.2 [\[ranges.syn\]](#), 26.7.23 [\[range.enumerate\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2023-03-30 **Last modified:** 2023-05-24

Priority: Not Prioritized

View other [active issues](#) in `[ranges.syn]`.

View all other [issues](#) in `[ranges.syn]`.

Discussion:

Originally editorial issue [Editorial issue #6151](#).

The *template-head* of `ranges::enumerate_view` in the header synopsis is different from those in the class synopses of itself and its iterator/sentinel pair.

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 26.2 [\[ranges.syn\]](#) as indicated:

```
[...]
// 26.7.23 \[range.enumerate\], enumerate view
template<view V input_range View>
    requires see below view<View>
class enumerate_view;
[...]
```

[3915](#). Redundant paragraph about expression variations

Section: 26.4.2 [\[range.range\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2023-04-01 **Last modified:** 2023-05-25

Priority: Not Prioritized

View all other [issues](#) in [\[range.range\]](#).

Discussion:

Originally editorial issue [Editorial issue #4431](#).

Expression variations kick in for "an expression that is non-modifying for some constant lvalue operand", but `std::ranges::range's` is an non-constant lvalue, so 26.4.2 [\[range.range\]](#) p2 is redundant.

I suppose that the change that clarified the template parameters' cv-qualification for purposes of equality-preservation and requiring additional variations happened concurrently with the change of `std::ranges::range's` operand from a forwarding reference to a non-constant lvalue reference.

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 26.2 [\[ranges.syn\]](#) as indicated:

-1- The range concept defines the requirements of a type that allows iteration over its elements by providing an iterator and sentinel that denote the elements of the range.

```
template<class T>
concept range =
    requires(T& t) {
        ranges::begin(t); // sometimes equality-preserving (see below)
        ranges::end(t);
    };

```

~~-2- The required expressions `ranges::begin(t)` and `ranges::end(t)` of the range concept do not require implicit expression variations (18.2 [\[concepts.equality\]](#)).~~

-3- [...]

[3925](#). Concept `formattable's` definition is incorrect

Section: 22.14.6.2 [\[format.formattable\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Mark de Wever **Opened:** 2023-04-16 **Last modified:** 2023-05-24

Priority: Not Prioritized

View other [active issues](#) in [format.formattable].

View all other [issues](#) in [format.formattable].

Discussion:

LWG [3631](#) modified the formattable concept. The new wording contains a small issue: `basic_format_context` requires two template arguments, but only one is provided.

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 22.14.6.2 [\[format.formattable\]](#) as indicated:

-1- Let *fmt-iter-for*<charT> be an unspecified type that models `output_iterator<const charT&>` (25.3.4.10 [\[iterator.concept.output\]](#)).

```
[...]
template<class T, class charT>
    concept formattable =
        formattable-with<remove_reference_t<T>, basic_format_context<fmt-iter-for<charT>, charT>>>;
```

[3927](#). Unclear preconditions for operator[] for sequence containers

Section: 24.2.4 [\[sequence.reqmts\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2023-04-24 **Last modified:** 2023-05-24

Priority: Not Prioritized

View other [active issues](#) in [sequence.reqmts].

View all other [issues](#) in [sequence.reqmts].

Discussion:

In 24.2.4 [\[sequence.reqmts\]](#)/118, we specify what `a[n]` means for a sequence container `a`, but we don't state that it actually has preconditions, other than implied ones.

When we want to use implied preconditions, we can actually get them by using the "*Effects: Equivalent to...*" wording.

[2023-05-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4944](#).

1. Modify 24.2.4 [\[sequence.reqmts\]](#) as indicated:

```
a[n]
```

-118- *Result:* reference; `const_reference` for constant `a`

-119- ~~Returns~~ *Effects:* **Equivalent to:** `return *(a.begin() + n);`

-120- *Remarks:* Required for `basic_string`, `array`, `deque`, and `vector`.

[3935](#). `template<class X> constexpr complex& operator=(const complex<X>&)` has no specification

Section: 28.4.3 [\[complex\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krügler **Opened:** 2023-05-21 **Last modified:** 2023-06-01

Priority: Not Prioritized

View other [active issues](#) in [complex].

View all other [issues](#) in [complex].

Discussion:

The class template `complex` synopsis in 28.4.3 [\[complex\]](#) shows the following member function template:

```
template<class X> constexpr complex& operator= (const complex<X>&);
```

but does not specify its semantics. This affects a code example such as the following one:

```
#include <complex>

int main()
{
    std::complex<double> zd(1, 1);
    std::complex<float> zf(2, 0);
    zd = zf;
}
```

This problem exists since the 1998 version of the standard (at that time this was declared in subclause [lib.complex]), and even though this looks like a "copy-assignment-like" operation, its effects aren't implied by our general 16.3.3.4 [\[functions.within.classes\]](#) wording (a function template is never considered as copy assignment operator, see 11.4.6 [\[class.copy.assign\]](#)).

It should be pointed out that the refactoring done by [P1467R9](#) had caused some other members to become finally specified that were not specified before, but in addition to LWG [3934](#)'s observation about the missing specification of the assignment operator taking a value type as parameter, this is now an additional currently unspecified member, but unaffected by any decision on LWG [3933](#).

[2023-06-01; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Add a new prototype specification between the current paragraphs 8 and 9 of 28.4.5 [\[complex.member.ops\]](#) as indicated:

[Drafting note: Similar to the proposed wording for LWG [3934](#), the wording form used below intentionally deviates from the rest of the [complex.member.ops] wording forms, because it seems much simpler and clearer to follow the wording forms used that specify the effects of `imag` and `real` functions plus borrowing relevant parts from 28.4.4 [\[complex.members\]](#) p2.]

```
constexpr complex& operator/=(const T& rhs);
```

```
[...]
```

```
template<class X> constexpr complex& operator=(const complex<X>& rhs);
```

-?- Effects: Assigns the value `rhs.real()` to the real part and the value `rhs.imag()` to the imaginary part of the complex value `*this`.

-?- Returns: `*this`.

```
template<class X> constexpr complex& operator+=(const complex<X>& rhs);
```

```
[...]
```

[3938](#). Cannot use `std::expected` monadic ops with move-only `error_type`

Section: 22.8.6.7 [\[expected.object.monadic\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2023-05-25 **Last modified:** 2023-06-01

Priority: Not Prioritized

View all other [issues](#) in [\[expected.object.monadic\]](#).

Discussion:

The monadic ops for `std::expected` are specified in terms of calls to `value()` and `error()`, but LWG [3843](#) ("`std::expected<T,E>::value()`& assumes E is copy constructible") added additional *Mandates* requirements to `value()`. This means that you can never call `value()` for a move-only `error_type`, even the overloads of `value()` with `rvalue` ref-qualifiers.

The changes to `value()` are because it needs to be able to throw a `bad_expected_access<E>` which requires a copyable E. But in the monadic ops we know it can't throw, because we always check. All the monadic ops are of the form:

```
if (has_value())
    do something with value();
else
    do something with error();
```

We know that `value()` won't throw here, but because we use "*Effects*: Equivalent to ..." the requirement for E to be copyable is inherited from `value()`.

Should we have changed the monadic ops to use `operator*()` instead of `value()`? For example, for the first and `_then` overloads the change would be:

-4- *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), value()**this);
else
    return U(unexpect, error());
```

[2023-06-01; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. For each *Effects*: element in 22.8.6.7 [\[expected.object.monadic\]](#), replace `value()` with `**this` as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let U be `remove_cvref_t<invoke_result_t<F, decltype(value()**this)>>`.

-2- *Constraints*: `is_constructible_v<E, decltype(error())>` is true.

-3- *Mandates*: U is a specialization of `expected` and `is_same_v<U::error_type, E>` is true.

-4- *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), value()**this);
else
    return U(unexpect, error());
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

-5- Let U be `remove_cvref_t<invoke_result_t<F, decltype(std::move(value()**this))>>`.

-6- *Constraints*: `is_constructible_v<E, decltype(std::move(error()))>` is true.

-7- *Mandates*: `U` is a specialization of `expected` and `is_same_v<U::error_type, E>` is true.

-8- *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), std::move(value()**this));
else
    return U(unexpect, std::move(error()));
```

```
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) const &;
```

-9- Let `G` be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

-10- *Constraints*: `is_constructible_v<T, decltype(value()**this)>` is true.

-11- *Mandates*: `G` is a specialization of `expected` and `is_same_v<G::value_type, T>` is true.

-12- *Effects*: Equivalent to:

```
if (has_value())
    return G(in_place, value()**this);
else
    return invoke(std::forward<F>(f), error());
```

```
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &&;
```

-13- Let `G` be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

-14- *Constraints*: `is_constructible_v<T, decltype(std::move(value()**this))>` is true.

-15- *Mandates*: `G` is a specialization of `expected` and `is_same_v<G::value_type, T>` is true.

-16- *Effects*: Equivalent to:

```
if (has_value())
    return G(in_place, std::move(value()**this));
else
    return invoke(std::forward<F>(f), std::move(error()));
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

-17- Let `U` be `remove_cv_t<invoke_result_t<F, decltype(value()**this)>>`.

-18- *Constraints*: `is_constructible_v<E, decltype(error())>` is true.

-19- *Mandates*: `U` is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f), value()**this));
```

is well-formed.

-20- *Effects*:

(20.1) — If `has_value()` is false, returns `expected<U, E>(unexpect, error())`.

(20.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose `has_val` member is true and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), value()**this)`.

(20.3) — Otherwise, evaluates `invoke(std::forward<F>(f), value()**this)` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
```



```
template<class F> constexpr auto transform(F&& f) const &&;
```

-21- Let U be `remove_cv_t<invoke_result_t<F, decltype(std::move(value())**this)>>`.

-22- *Constraints*: `is_constructible_v<E, decltype(std::move(error()))>` is true.

-23- *Mandates*: U is a valid value type for expected. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f), std::move(value())**this));
```

is well-formed for some invented variable u.

[Drafting Note: The removal of "for some invented variable u" in paragraph 23 is a drive-by fix for consistency with paragraphs 19, 27 and 31.]

-24- *Effects*:

(24.1) — If `has_value()` is false, returns `expected<U, E>(unexpected, error())`.

(24.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose `has_val` member is true and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(value())**this)`.

(24.3) — Otherwise, evaluates `invoke(std::forward<F>(f), std::move(value())**this)` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

-25- Let G be `remove_cv_t<invoke_result_t<F, decltype(error())>>`.

-26- *Constraints*: `is_constructible_v<T, decltype(value())**this>` is true.

-27- *Mandates*: G is a valid template argument for `unexpected` ([`unexpected.un.general`]) and the declaration

```
G g(invoke(std::forward<F>(f), error()));
```

is well-formed.

-28- *Returns*: If `has_value()` is true, `expected<T, G>(in_place, value())**this`; otherwise, an `expected<T, G>` object whose `has_val` member is false and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

-29- Let G be `remove_cv_t<invoke_result_t<F, decltype(std::move(error()))>>`.

-30- *Constraints*: `is_constructible_v<T, decltype(std::move(value())**this)>` is true.

-31- *Mandates*: G is a valid template argument for `unexpected` ([`unexpected.un.general`]) and the declaration

```
G g(invoke(std::forward<F>(f), std::move(error())));
```

is well-formed.

-32- *Returns*: If `has_value()` is true, `expected<T, G>(in_place, std::move(value())**this)`; otherwise, an `expected<T, G>` object whose `has_val` member is false and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

[3940](#). `std::expected<void, E>::value()` also needs E to be copy constructible

Section: 22.8.7.6 [[expected.void.obs](#)] **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-05-26 **Last modified:** 2023-06-01

Priority: Not Prioritized

Discussion:

LWG [3843](#) added *Mandates:* to `std::expected::value`, but the similar handling is missing for `expected<cv void, E>`.

[2023-06-01; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 22.8.7.6 [\[expected.void.obs\]](#) as indicated:

```
constexpr void value() const &;
```

~~-?- *Mandates:* is copy constructible v<E> is true.~~

-3- *Throws:* `bad_expected_access(error())` if `has_value()` is false.

```
constexpr void value() &&;
```

~~-?- *Mandates:* is copy constructible v<E> is true and is move constructible v<E> is true.~~

-4- *Throws:* `bad_expected_access(std::move(error()))` if `has_value()` is false.